

We Are A Team

搜索 AI提问 评论 笔记

题目描述

总共有 n 个人在机房，每个人有一个标号 ($1 \leq \text{标号} \leq n$)，他们分成了多个团队，需要你根据收到的 m 条消息判定指定的两个人是否在一个团队中，具体的：

- 1. 消息构成为 $a\ b\ c$ ，整数 a 、 b 分别代表两个人的标号，整数 c 代表指令
- 2. $c == 0$ 代表 a 和 b 在一个团队内
- 3. $c == 1$ 代表需要判定 a 和 b 的关系，如果 a 和 b 是一个团队，输出一行 'we are a team', 如果不是，输出一行 'we are not a team'
- 4. c 为其他值，或当前行 a 或 b 超出 $1 \sim n$ 的范围，输出 'da pian zi'

输入描述

- 1. 第一行包含两个整数 $n, m (1 \leq n, m < 100000)$, 分别表示有 n 个人和 m 条消息
- 2. 随后的 m 行，每行一条消息，消息格式为： $a\ b\ c (1 \leq a, b \leq n, 0 \leq c \leq 1)$

输出描述

- 1. $c == 1$, 根据 a 和 b 是否在一个团队中输出一行字符串，在一个团队中输出 'we are a team', 不在一个团队中输出 'we are not a team'
- 2. c 为其他值，或当前行 a 或 b 的标号小于 1 或者大于 n 时，输出字符串 'da pian zi'
- 3. 如果第一行 n 和 m 的值超出约定的范围时，输出字符串 "NULL"。

用例

输入	5 7 1 2 0 4 5 0 2 3 0
----	--------------------------------

	1 2 1 2 3 1 4 5 1 1 5 1
输出	we are a team we are a team we are a team we are not a team
说明	无

输入	5 6 1 2 0 1 2 1 1 5 0 2 3 1 2 5 1 1 3 2
输出	we are a team we are not a team we are a team da pian zi
说明	无

题目解析

本题可以使用 **并查集** 建立 a,b 的关联关系。

2023.09.19

本题c == 1判断a,b是否处于一个团队时，本题判断范围存在歧义，当前有两种思路：

- 1、仅限判断前面行中建立的关系中，a,b是否处于同一团队
- 2、不限于前面行，而是等所有行输入完后，才判断a,b是否处于同一团队

下面对这两种思路都进行实现下，代码上差别不大，考试时可以都试下

c==1时 仅在当前行前面建立的关系中，判断a,b是否处于同一团队

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const [n, m] = (await readline()).split(" ").map(Number);
7
8      const msgs = [];
9      for (let i = 0; i < m; i++) {
10         msgs.push((await readline()).split(" ").map(Number));
11     }
12
13     getResult(msgs, n, m);
14 })();
15
16 function getResult(msgs, n, m) {
17     // 如果第一行 n 和 m 的值超出约定的范围时，输出字符串“NULL”。
18     // 1<=n,m<100000
19     if (n < 1 || n >= 100000 || m < 1 || m >= 100000) return console.log("NULL");
20
21     const ufs = new UnionFindSet(n + 1);
22
23     msgs.forEach((msg) => {
24         const [a, b, c] = msg;
25
26         if (a < 1 || a > n || b < 1 || b > n) {
```

运行

```

27 | // 当前行 a 或 b 的标号小于 1 或者大于 n 时, 输出字符串 'da pian zi'
28 | return console.log("da pian zi");
29 | }
30 |
31 | if (c === 0) {
32 |     // c == 0 代表 a 和 b 在一个团队内
33 |     ufs.union(a, b);
34 | } else if (c === 1) {
35 |     // c == 1 代表需要判定 a 和 b 的关系, 如果 a 和 b 是一个团队, 输出一行 'we are a team', 如果不是, 输出一行 'we are not a team'
36 |     console.log(
37 |         ufs.find(a) === ufs.find(b) ? "we are a team" : "we are not a team"
38 |     );
39 | } else {
40 |     // c 为其他值, 输出字符串 'da pian zi'
41 |     console.log("da pian zi");
42 | }
43 | });
44 | }
45 |
46 | // 并查集实现
47 | class UnionFindSet {
48 |     constructor(n) {
49 |         this.fa = new Array(n).fill(0).map((_, idx) => idx);
50 |     }
51 |
52 |     find(x) {
53 |         if (this.fa[x] !== x) {
54 |             this.fa[x] = this.find(this.fa[x]);
55 |             return this.fa[x];
56 |         }
57 |         return x;
58 |     }
59 |
60 |     union(x, y) {
61 |         let x_fa = this.find(x);
62 |         let y_fa = this.find(y);
63 |

```

```

64 |         if (x_fa != y_fa) {
65 |             this.fa[y_fa] = x_fa;
66 |         }
67 |     }
68 | }

```

Java算法源码

```

1 | import java.util.Scanner;
2 |
3 | public class Main {
4 |     public static void main(String[] args) {
5 |         Scanner sc = new Scanner(System.in);
6 |
7 |         int n = sc.nextInt();
8 |         int m = sc.nextInt();
9 |
10 |        int[][] msgs = new int[m][3];
11 |        for (int i = 0; i < m; i++) {
12 |            msgs[i][0] = sc.nextInt();
13 |            msgs[i][1] = sc.nextInt();
14 |            msgs[i][2] = sc.nextInt();
15 |        }
16 |
17 |        getResult(msgs, n, m);
18 |    }
19 |
20 |    public static void getResult(int[][] msgs, int n, int m) {
21 |        // 如果第一行 n 和 m 的值超出约定的范围时，输出字符串“Null”。
22 |        // 1<=n,m<100000
23 |        if (n < 1 || n >= 100000 || m < 1 || m >= 100000) {
24 |            System.out.println("NULL");
25 |            return;
26 |        }

```

```

27 |
28 |     UnionFindSet ufs = new UnionFindSet(n + 1);
29 |
30 |     for (int[] msg : msgs) {
31 |         int a = msg[0], b = msg[1], c = msg[2];
32 |
33 |         if (a < 1 || a > n || b < 1 || b > n) {
34 |             // 当前行 a 或 b 的标号小于 1 或者大于 n 时, 输出字符串'da pian zi'
35 |             System.out.println("da pian zi");
36 |             continue;
37 |         }
38 |
39 |         if (c == 0) {
40 |             // c == 0 代表 a 和 b 在一个团队内
41 |             ufs.union(a, b);
42 |         } else if (c == 1) {
43 |             // c == 1 代表需要判定 a 和 b 的关系, 如果 a 和 b 是一个团队, 输出一行'we are a team', 如果不是, 输出一行'we are not a team'
44 |             System.out.println(ufs.find(a) == ufs.find(b) ? "we are a team" : "we are not a team");
45 |         } else {
46 |             // c 为其他值, 输出字符串'da pian zi'
47 |             System.out.println("da pian zi");
48 |         }
49 |     }
50 | }
51 |
52 |
53 | // 并查集实现
54 | class UnionFindSet {
55 |     int[] fa;
56 |
57 |     public UnionFindSet(int n) {
58 |         this.fa = new int[n];
59 |         for (int i = 0; i < n; i++) fa[i] = i;
60 |     }
61 |
62 |     public int find(int x) {
63 |         if (this.fa[x] != x) {
64 |             return this.fa[x] = this.find(this.fa[x]);

```

```
65     }
66     return x;
67 }
68
69 public void union(int x, int y) {
70     int x_fa = this.find(x);
71     int y_fa = this.find(y);
72
73     if (x_fa != y_fa) {
74         this.fa[y_fa] = x_fa;
75     }
76 }
77 }
```

Python算法源码

```
1  # 输入获取
2  n, m = map(int, input().split())
3  msgs = [list(map(int, input().split())) for _ in range(m)]
4
5
6  # 并查集实现
7  class UnionFindSet:
8      def __init__(self, n):
9          self.fa = [i for i in range(n)]
10
11      def find(self, x):
12          if self.fa[x] != x:
13              self.fa[x] = self.find(self.fa[x])
14          return self.fa[x]
15      return x
16
17      def union(self, x, y):
18          x_fa = self.find(x)
19          y_fa = self.find(y)
```

```

20 |
21 |         if x_fa != y_fa:
22 |             self.fa[y_fa] = x_fa
23 |
24 |
25 | # 算法入口
26 | def getResult():
27 |     # 如果第一行  $n$  和  $m$  的值超出约定的范围时, 输出字符串"Null"。
28 |     #  $1 \leq n, m < 100000$ 
29 |     if n < 1 or n >= 100000 or m < 1 or m >= 100000:
30 |         print("NULL")
31 |         return
32 |
33 |     ufs = UnionFindSet(n + 1)
34 |
35 |     for a, b, c in msgs:
36 |         if a < 1 or a > n or b < 1 or b > n:
37 |             # 当前行  $a$  或  $b$  的标号小于 1 或者大于  $n$  时, 输出字符串'da pian zi'
38 |             print("da pian zi")
39 |             continue
40 |
41 |         if c == 0:
42 |             #  $c == 0$  代表  $a$  和  $b$  在一个团队内
43 |             ufs.union(a, b)
44 |         elif c == 1:
45 |             #  $c == 1$  代表需要判定  $a$  和  $b$  的关系, 如果  $a$  和  $b$  是一个团队, 输出一行'we are a team', 如果不是, 输出一行'we are not a team'
46 |             print("we are a team" if ufs.find(a) == ufs.find(b) else "we are not a team")
47 |         else:
48 |             #  $c$  为其他值, 输出字符串'da pian zi'
49 |             print("da pian zi")
50 |
51 |
52 | # 算法调用
53 | getResult()

```


C算法源码

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /** 并查集定义 **/
5  typedef struct {
6      int *fa;
7      int count;
8  } UFS;
9
10 UFS *new_UFS(int n) {
11     UFS *ufs = (UFS *) malloc(sizeof(UFS));
12
13     ufs->fa = (int *) malloc(sizeof(int) * n);
14     for (int i = 0; i < n; i++) {
15         ufs->fa[i] = i;
16     }
17
18     ufs->count = n;
19
20     return ufs;
21 }
22
23 int find_UFS(UFS *ufs, int x) {
24     if (x != ufs->fa[x]) {
25         ufs->fa[x] = find_UFS(ufs, ufs->fa[x]);
26         return ufs->fa[x];
27     }
28     return x;
29 }
30
31 void union_UFS(UFS *ufs, int x, int y) {
32     int x_fa = find_UFS(ufs, x);
33     int y_fa = find_UFS(ufs, y);
34
35     if (x_fa != y_fa) {
```

```

36         ufs->fa[y_fa] = x_fa;
37         ufs->count--;
38     }
39 }
40
41 int main() {
42     int n, m;
43     scanf("%d %d", &n, &m);
44
45     int msgs[m][3];
46     for (int i = 0; i < m; i++) {
47         scanf("%d %d %d", &msgs[i][0], &msgs[i][1], &msgs[i][2]);
48     }
49
50     // 如果第一行 n 和 m 的值超出约定的范围时, 输出字符串"Null"。
51     // 1<=n,m<100000
52     if (n < 1 || n >= 100000 || m < 1 || m >= 100000) {
53         puts("NULL");
54         return 0;
55     }
56
57     UFS *ufs = new_UFS(n + 1);
58
59     for (int i = 0; i < m; i++) {
60         int a = msgs[i][0], b = msgs[i][1], c = msgs[i][2];
61
62         if(a < 1 || a > n || b < 1 || b > n) {
63             // 当前行 a 或 b 的标号小于 1 或者大于 n 时, 输出字符串'da pian zi'
64             puts("da pian zi");
65             continue;
66         }
67
68         if(c == 0) {
69             // c == 0 代表 a 和 b 在一个团队内
70             union_UFS(ufs, a, b);
71         } else if(c == 1) {
72             // c == 1 代表需要判定 a 和 b 的关系, 如果 a 和 b 是一个团队, 输出一行'we are a team',如果不是, 输出一行'we are not a team'

```

```

73         puts(find_UFS(ufs, a) == find_UFS(ufs, b) ? "we are a team" : "we are not a team");
74     } else {
75         // c 为其他值，输出字符串‘da pian zi’
76         puts("da pian zi");
77     }
78 }
79
80 return 0;
81 }

```

C++算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  // 并查集实现
6  class UnionFindSet {
7  public:
8      int *fa;
9
10     explicit UnionFindSet(int n) {
11         fa = new int[n];
12         for (int i = 0; i < n; i++) fa[i] = i;
13     };
14
15     int find(int x) {
16         if (x != fa[x]) {
17             return (fa[x] = find(fa[x]));
18         }
19         return x;
20     };
21
22     void merge(int x, int y) {
23         int x_fa = find(x);

```

运行

```

24         int y_fa = find(y); 25
26         if (x_fa != y_fa) {
27             fa[y_fa] = x_fa;
28         }
29     };
30 };
31
32 int main() {
33     int n, m;
34     cin >> n >> m;
35
36     // 如果第一行 n 和 m 的值超出约定的范围时, 输出字符串"NULL"。
37     // 1<=n,m<100000
38     if (n < 1 || n >= 100000 || m < 1 || m >= 100000) {
39         cout << "NULL" << endl;
40         return 0;
41     }
42
43     UnionFindSet ufs(n + 1);
44
45     for (int i = 0; i < m; i++) {
46         int a, b, c;
47         cin >> a >> b >> c;
48
49         if (a < 1 || a > n || b < 1 || b > n) {
50             // 当前行 a 或 b 的标号小于 1 或者大于 n 时, 输出字符串'da pian zi'
51             cout << "da pian zi" << endl;
52             continue;
53         }
54
55         if (c == 0) {
56             // c == 0 代表 a 和 b 在一个团队内
57             ufs.merge(a, b);
58         } else if (c == 1) {
59             // c == 1 代表需要判定 a 和 b 的关系, 如果 a 和 b 是一个团队, 输出一行'we are a team', 如果不是, 输出一行'we are not a team'
60             if (ufs.find(a) == ufs.find(b)) {
61                 cout << "we are a team" << endl;

```

```

62         } else {
63             cout << "we are not a team" << endl;
64         }
65     } else {
66         // c 为其他值，输出字符串‘da pian zi’
67         cout << "da pian zi" << endl;
68     }
69 }
70
71 return 0;
72 }

```

c==1时 等所有行建立的关系都完成后，再判断a,b是否处于同一团队

JS算法源码

```

1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const [n, m] = (await readline()).split(" ").map(Number);
7
8      const msgs = [];
9      for (let i = 0; i < m; i++) {
10         msgs.push((await readline()).split(" ").map(Number));
11     }
12
13     getResult(msgs, n, m);
14 })();
15
16 function getResult(msgs, n, m) {
17     // 如果第一行 n 和 m 的值超出约定的范围时，输出字符串“Null”。

```

```

18 // 1<=n,m<100000 | if (n < 1 || n >= 100000 || m < 1 || m >= 100000) return console.log("NULL");
19
20
21 const ufs = new UnionFindSet(n + 1);
22
23 // 过滤出 c == 0 的, 且非异常的行, 提前构建出所有关系
24 msgs
25   .filter(([a, b, c]) => c == 0 && a >= 1 && a <= n && b >= 1 && b <= n)
26   .forEach(([a, b]) => ufs.union(a, b));
27
28 // 处理 c 不为0的其他情况
29 msgs.forEach((msg) => {
30   const [a, b, c] = msg;
31
32   if (a < 1 || a > n || b < 1 || b > n) {
33     // 当前行 a 或 b 的标号小于 1 或者大于 n 时, 输出字符串 'da pian zi'
34     return console.log("da pian zi");
35   }
36
37   if (c === 0) {
38   } else if (c === 1) {
39     // c == 1 代表需要判定 a 和 b 的关系, 如果 a 和 b 是一个团队, 输出一行 'we are a team', 如果不是, 输出一行 'we are not a team'
40     console.log(
41       ufs.find(a) === ufs.find(b) ? "we are a team" : "we are not a team"
42     );
43   } else {
44     // c 为其他值, 输出字符串 'da pian zi'
45     console.log("da pian zi");
46   }
47 });
48 }
49
50 // 并查集实现
51 class UnionFindSet {
52   constructor(n) {
53     this.fa = new Array(n).fill(0).map((_, idx) => idx);
54   }
55

```

```

56 | find(x) {
    |     57 |     if (this.fa[x] !== x) {
58 |         this.fa[x] = this.find(this.fa[x]);
59 |         return this.fa[x];
60 |     }
61 |     return x;
62 | }
63 |
64 | union(x, y) {
65 |     let x_fa = this.find(x);
66 |     let y_fa = this.find(y);
67 |
68 |     if (x_fa !== y_fa) {
69 |         this.fa[y_fa] = x_fa;
70 |     }
71 | }
72 | }

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         int n = sc.nextInt();
9 |         int m = sc.nextInt();
10 |
11 |         int[][] msgs = new int[m][3];
12 |         for (int i = 0; i < m; i++) {
13 |             msgs[i][0] = sc.nextInt();
14 |             msgs[i][1] = sc.nextInt();
15 |             msgs[i][2] = sc.nextInt();
16 |         }

```

```

17
18 |     getResult(msgs, n, m);
19 | }
20
21 public static void getResult(int[][] msgs, int n, int m) {
22     // 如果第一行 n 和 m 的值超出约定的范围时，输出字符串“Null”。
23     // 1<=n,m<100000
24     if (n < 1 || n >= 100000 || m < 1 || m >= 100000) {
25         System.out.println("NULL");
26         return;
27     }
28
29     UnionFindSet ufs = new UnionFindSet(n + 1);
30
31     // 过滤出 c == 0 的，且非异常的行，提前构建出所有关系
32     Arrays.stream(msgs)
33         .filter(msg -> msg[2] == 0 && msg[0] >= 1 && msg[0] <= n && msg[1] >= 1 && msg[1] <= n)
34         .forEach(msg -> ufs.union(msg[0], msg[1]));
35
36     // 处理 c 不为0的其他情况
37     for (int[] msg : msgs) {
38         int a = msg[0], b = msg[1], c = msg[2];
39
40         if (a < 1 || a > n || b < 1 || b > n) {
41             // 当前行 a 或 b 的标号小于 1 或者大于 n 时，输出字符串‘da pian zi’
42             System.out.println("da pian zi");
43             continue;
44         }
45
46         if (c == 0) {
47         } else if (c == 1) {
48             // c == 1 代表需要判定 a 和 b 的关系，如果 a 和 b 是一个团队，输出一行‘we are a team’，如果不是，输出一行‘we are not a team’
49             System.out.println(ufs.find(a) == ufs.find(b) ? "we are a team" : "we are not a team");
50         } else {
51             // c 为其他值，输出字符串‘da pian zi’
52             System.out.println("da pian zi");
53         }

```



```

54     }55 |   }
56 }
57
58 // 并查集实现
59 class UnionFindSet {
60     int[] fa;
61
62     public UnionFindSet(int n) {
63         this.fa = new int[n];
64         for (int i = 0; i < n; i++) fa[i] = i;
65     }
66
67     public int find(int x) {
68         if (this.fa[x] != x) {
69             return this.fa[x] = this.find(this.fa[x]);
70         }
71         return x;
72     }
73
74     public void union(int x, int y) {
75         int x_fa = this.find(x);
76         int y_fa = this.find(y);
77
78         if (x_fa != y_fa) {
79             this.fa[y_fa] = x_fa;
80         }
81     }
82 }

```

Python算法源码

```

1 # 输入获取
2 n, m = map(int, input().split())
3 msgs = [list(map(int, input().split())) for _ in range(m)]
4
5

```

```
6 # 并查集实现
7 class UnionFindSet:
8     def __init__(self, n):
9         self.fa = [i for i in range(n)]
10
11     def find(self, x):
12         if self.fa[x] != x:
13             self.fa[x] = self.find(self.fa[x])
14         return self.fa[x]
15     return x
16
17     def union(self, x, y):
18         x_fa = self.find(x)
19         y_fa = self.find(y)
20
21         if x_fa != y_fa:
22             self.fa[y_fa] = x_fa
23
24
25 # 算法入口
26 def getResult():
27     # 如果第一行 n 和 m 的值超出约定的范围时, 输出字符串"NULL"。
28     # 1<=n,m<100000
29     if n < 1 or n >= 100000 or m < 1 or m >= 100000:
30         print("NULL")
31         return
32
33     ufs = UnionFindSet(n + 1)
34
35     for a, b, c in filter(lambda msg: msg[2] == 0 and 1 <= msg[0] <= n and 1 <= msg[1] <= n, msgs):
36         ufs.union(a, b)
37
38     for a, b, c in msgs:
39         if a < 1 or a > n or b < 1 or b > n:
40             # 当前行 a 或 b 的标号小于 1 或者大于 n 时, 输出字符串'da pian zi'
41             print("da pian zi")
42             continue
```

```

43 |
44 |         if c == 0:
45 |             pass
46 |         elif c == 1:
47 |             # c == 1 代表需要判定 a 和 b 的关系, 如果 a 和 b 是一个团队, 输出一行 'we are a team', 如果不是, 输出一行 'we are not a team'
48 |             print("we are a team" if ufs.find(a) == ufs.find(b) else "we are not a team")
49 |         else:
50 |             # c 为其他值, 输出字符串 'da pian zi'
51 |             print("da pian zi")
52 |
53 |
54 | # 算法调用
55 | getResult()

```

C算法源码

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /** 并查集定义 */
5  typedef struct {
6      int *fa;
7      int count;
8  } UFS;
9
10 UFS *new_UFS(int n) {
11     UFS *ufs = (UFS *) malloc(sizeof(UFS));
12
13     ufs->fa = (int *) malloc(sizeof(int) * n);
14     for (int i = 0; i < n; i++) {
15         ufs->fa[i] = i;
16     }
17
18     ufs->count = n;
19
20     return ufs;

```

```

21 | }22 |
23 | int find_UFS(UFS *ufs, int x) {
24 |     if (x != ufs->fa[x]) {
25 |         ufs->fa[x] = find_UFS(ufs, ufs->fa[x]);
26 |         return ufs->fa[x];
27 |     }
28 |     return x;
29 | }
30 |
31 | void union_UFS(UFS *ufs, int x, int y) {
32 |     int x_fa = find_UFS(ufs, x);
33 |     int y_fa = find_UFS(ufs, y);
34 |
35 |     if (x_fa != y_fa) {
36 |         ufs->fa[y_fa] = x_fa;
37 |         ufs->count--;
38 |     }
39 | }
40 |
41 | int main() {
42 |     int n, m;
43 |     scanf("%d %d", &n, &m);
44 |
45 |     int msgs[m][3];
46 |     for (int i = 0; i < m; i++) {
47 |         scanf("%d %d %d", &msgs[i][0], &msgs[i][1], &msgs[i][2]);
48 |     }
49 |
50 |     // 如果第一行 n 和 m 的值超出约定的范围时，输出字符串“NULL”。
51 |     // 1<=n,m<100000
52 |     if (n < 1 || n >= 100000 || m < 1 || m >= 100000) {
53 |         puts("NULL");
54 |         return 0;
55 |     }
56 |
57 |     UFS *ufs = new_UFS(n + 1);
58 |

```

```

59 // 过滤出  $c == 0$  的, 且非异常的行, 提前构建出所有关系
60 for (int i = 0; i < m; i++) {
61     int a = msgs[i][0], b = msgs[i][1], c = msgs[i][2];
62
63     if (c == 0 && a >= 1 && a <= n && b >= 1 && b <= n) {
64         union_UFS(ufs, a, b);
65     }
66 }
67
68 // 处理  $c$  不为0的其他情况
69 for (int i = 0; i < m; i++) {
70     int a = msgs[i][0], b = msgs[i][1], c = msgs[i][2];
71
72     if (a < 1 || a > n || b < 1 || b > n) {
73         // 当前行  $a$  或  $b$  的标号小于 1 或者大于  $n$  时, 输出字符串 'da pian zi'
74         puts("da pian zi");
75         continue;
76     }
77
78     if(c == 0) continue;
79
80     if (c == 1) {
81         //  $c == 1$  代表需要判定  $a$  和  $b$  的关系, 如果  $a$  和  $b$  是一个团队, 输出一行 'we are a team', 如果不是, 输出一行 'we are not a team'
82         puts(find_UFS(ufs, a) == find_UFS(ufs, b) ? "we are a team" : "we are not a team");
83     } else {
84         //  $c$  为其他值, 输出字符串 'da pian zi'
85         puts("da pian zi");
86     }
87 }
88
89 return 0;
90 }

```

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | // 并查集实现
6 | class UnionFindSet {
7 | public:
8 |     int *fa;
9 |
10 |    explicit UnionFindSet(int n) {
11 |        fa = new int[n];
12 |        for (int i = 0; i < n; i++) fa[i] = i;
13 |    };
14 |
15 |    int find(int x) {
16 |        if (x != fa[x]) {
17 |            return (fa[x] = find(fa[x]));
18 |        }
19 |        return x;
20 |    };
21 |
22 |    void merge(int x, int y) {
23 |        int x_fa = find(x);
24 |        int y_fa = find(y);
25 |
26 |        if (x_fa != y_fa) {
27 |            fa[y_fa] = x_fa;
28 |        }
29 |    };
30 | };
31 |
32 | int main() {
33 |     int n, m;
34 |     cin >> n >> m;
35 |
36 |     vector<vector<int>> msgs(m, vector<int>(3));
37 |     for (int i = 0; i < m; i++) {

```

```

38     cin >> msgs[i][0] >> msgs[i][1] >> msgs[i][2]; 39     }
39
40
41 // 如果第一行  $n$  和  $m$  的值超出约定的范围时, 输出字符串 "Null"。
42 //  $1 \leq n, m < 100000$ 
43 if (n < 1 || n >= 100000 || m < 1 || m >= 100000) {
44     cout << "NULL" << endl;
45     return 0;
46 }
47
48 UnionFindSet ufs(n + 1);
49
50 // 过滤出  $c == 0$  的, 且非异常的行, 提前构建出所有关系
51 for (int i = 0; i < m; i++) {
52     int a = msgs[i][0], b = msgs[i][1], c = msgs[i][2];
53
54     if (c == 0 && a >= 1 && a <= n && b >= 1 && b <= n) {
55         ufs.merge(a, b);
56     }
57 }
58
59 // 处理  $c$  不为0的其他情况
60 for (int i = 0; i < m; i++) {
61     int a = msgs[i][0], b = msgs[i][1], c = msgs[i][2];
62
63     if (a < 1 || a > n || b < 1 || b > n) {
64         // 当前行  $a$  或  $b$  的标号小于 1 或者大于  $n$  时, 输出字符串 'da pian zi'
65         cout << "da pian zi" << endl;
66         continue;
67     }
68
69     if (c == 0) continue;
70
71     if (c == 1) {
72         //  $c == 1$  代表需要判定  $a$  和  $b$  的关系, 如果  $a$  和  $b$  是一个团队, 输出一行 'we are a team', 如果不是, 输出一行 'we are not a team'
73         if (ufs.find(a) == ufs.find(b)) {
74             cout << "we are a team" << endl;

```

```
75         } else {76             cout << "we are not a team" << endl;
77     }
78     } else {
79         // c 为其他值，输出字符串‘da pian zi’
80         cout << "da pian zi" << endl;
81     }
82 }
83
84 return 0;
85 }
```